AI Features - XiaoZhi Firmware AI Technology Integration Guide

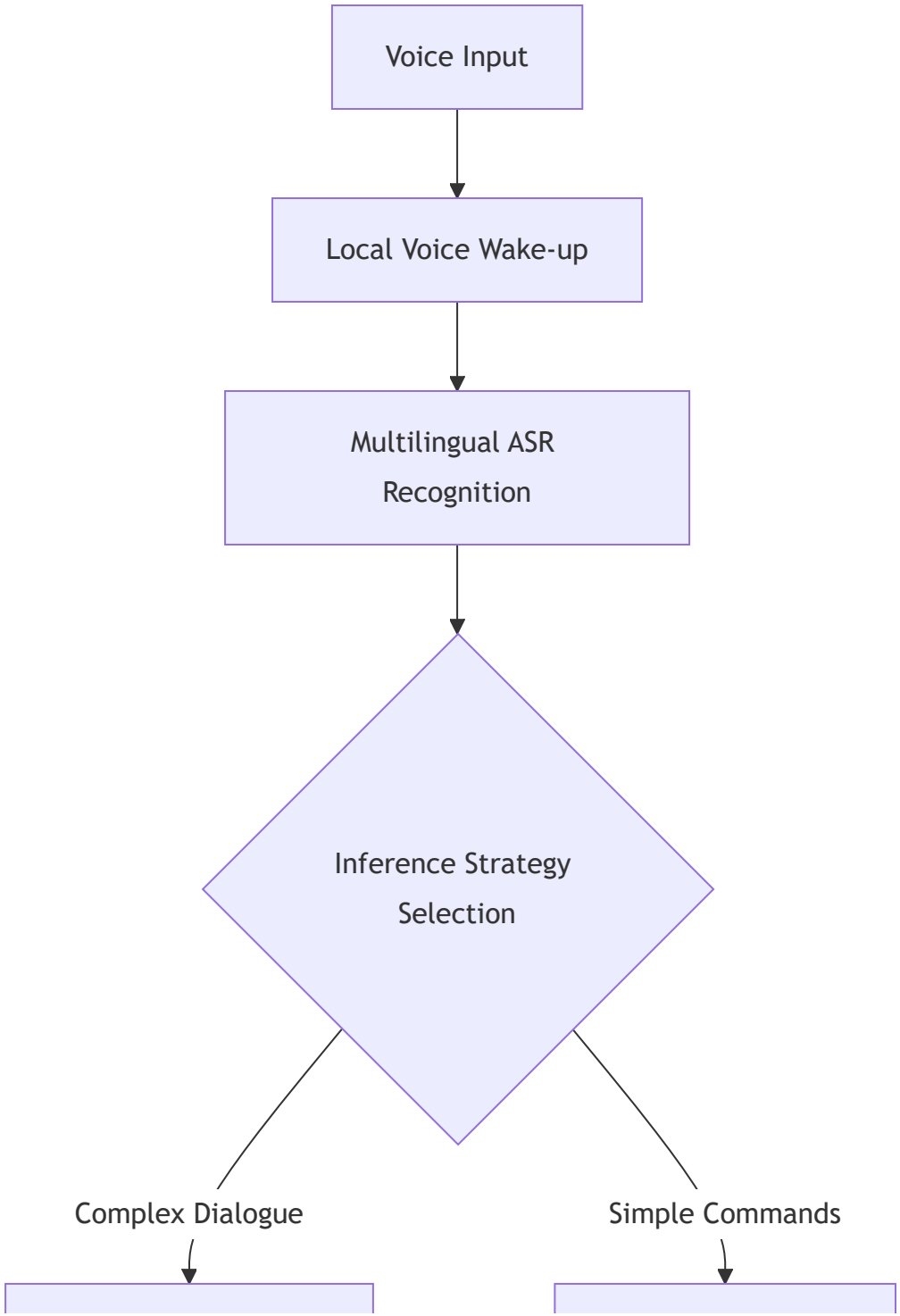
🤖 AI Features

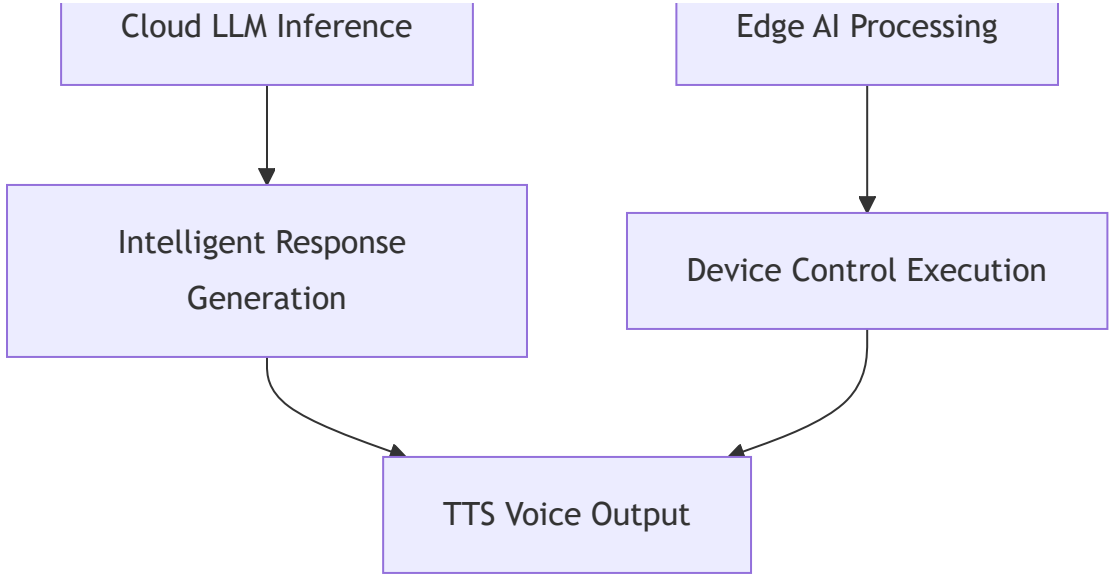
Learn how to implement voice interaction, AI model integration and smart control functions on the ESP32-S3 platform.

🎯 Core AI Architecture

ⓘ **Technology Positioning**: XiaoZhi AI = Edge Intelligence + Cloud LLM + Protocol-based IoT Control

🧠 Hybrid AI Inference Architecture


Voice Input
Local Voice Wake-up
Multilingual ASR Recognition
Inference Strategy Selection
Complex Dialogue
Simple Commands



🔥 Core AI Features Deep Dive

1️⃣ Offline Voice Wake-up AI

Technical Principle: Based on Espressif's official Wake Word Engine

- 🎙️ **Default Wake Word**: "你好小智" (Customizable with 26+ official wake words)
- ⚡ **Response Speed**: <200ms ultra-low latency wake-up
- 🔋 **Power Optimization**: Wake-up standby power consumption <5mA
- 🌐 **Offline Operation**: Completely local, no network connection required

🛠️ Wake Word Customization Development Guide

```
# ESP-IDF environment wake word configuration
idf.py menuconfig
# Navigate to: Component config > ESP Speech Recognition
# Select: Wake Word Model Selection
# Available words: "Hi ESP", "Alexa", "小智" and 26 other official vocabularies
```

Supported Wake Word List:

- Chinese: “你好小智”, “小智助手”, “智能管家”
- English: “Hi ESP”, “Hello World”, “Smart Home”
- Japanese: “コンニチワ”, “スマート”
- Korean: “안녕하세요”, “스마트”

2️⃣ Multilingual Intelligent Speech Recognition (ASR)

Technology Stack: Integrates industry-leading ASR engines

- 🗣️ **Supported Languages**: Chinese (Mandarin/Cantonese) | English | Japanese | Korean | Russian
- 🎯 **Recognition Accuracy**: Chinese recognition rate >95%, English recognition rate >93%
- 🔊 **Audio Format**: 16kHz sampling rate, 16-bit PCM encoding
- 🌍 **Offline/Online**: Hybrid mode, keywords offline, complex sentences online

> **v2.0 Globalization Upgrade**: Added 17 new languages, supporting 25+ languages in total, each with complete voice prompt audio resource package (OGG format)

🌐 Multilingual Localization Technical Architecture

Voice Resource Package Structure:

```
main/assets/locales/
├── zh-CN/           # Simplified Chinese
│   ├── 0.ogg ~ 9.ogg        # Number audio
│   ├── welcome.ogg          # Welcome prompt
│   ├── activation.ogg       # Activation success
│   ├── wificonfig.ogg       # Wi-Fi configuration
│   ├── upgrade.ogg          # Firmware upgrade
│   ├── err_pin.ogg          # PIN error
│   ├── err_reg.ogg          # Registration error
│   └── language.json        # Language metadata
├── en-US/           # English (same structure)
├── ja-JP/           # Japanese
├── bg-BG/           # Bulgarian (v2.0 new)
├── ca-ES/           # Catalan (v2.0 new)
├── da-DK/           # Danish (v2.0 new)
├── el-GR/           # Greek (v2.0 new)
├── fa-IR/           # Persian (v2.0 new)
├── fil-PH/          # Filipino (v2.0 new)
├── he-IL/           # Hebrew (v2.0 new)
├── hr-HR/           # Croatian (v2.0 new)
├── hu-HU/           # Hungarian (v2.0 new)
├── ms-MY/           # Malay (v2.0 new)
├── nb-NO/           # Norwegian (v2.0 new)
├── nl-NL/           # Dutch (v2.0 new)
├── sk-SK/           # Slovak (v2.0 new)
```

```
├── sl-SI/          # Slovenian (v2.0 new)
├── sr-RS/          # Serbian (v2.0 new)
└── sv-SE/          # Swedish (v2.0 new)
```

Technical Implementation:

- **Audio Format**: OGG Vorbis, 16kHz sampling rate
- **Storage Optimization**: ~150KB per language, compile on demand
- **Runtime Loading**: Load corresponding language package based on configuration at startup
- **Memory Usage**: ~50KB RAM for current language resources

Language Switching Configuration:

```
// sdkconfig configuration
CONFIG_XIAOZHI_LANGUAGE="zh-CN"  // Set default language

// menuconfig path
// Xiaozhi Assistant -> Language Settings -> Select Language
```

Supported Language Tiers:

- **Tier 1 (Priority Optimized)**: Chinese, English, Japanese, Russian
- **Tier 2 (Full Support)**: Spanish, French, German, Korean
- **Tier 3 (Basic Support)**: Other 17 newly added languages

⚠ **Performance Tip**: Complex multilingual mixed recognition requires cloud ASR support, stable network environment recommended

---

3️⃣ Large Language Model Integration & Inference

🚀 Supported Mainstream AI Large Models

AI Model	Provider	Inference Method	Special Capabilities	Access Cost
DeepSeek-V3	DeepSeek	Cloud API	Math Reasoning/Code Generation	Low Cost
Qwen-Max	Alibaba Cloud	Cloud API	Chinese Understanding/Multimodal	Medium
Doubao-Pro	ByteDance	Cloud API	Dialogue Generation/Creative Writing	Medium
ChatGPT-4o	OpenAI	Cloud API	General Intelligence/Logic Reasoning	High Cost
Gemini	Google	Cloud API	Multimodal/Real-time Interaction	Medium-High

🧩 Edge AI Inference Capabilities

Lightweight Model Support (Planned Features):

- 🧮 **TensorFlow Lite**: Support for quantized lightweight models
- 🔧 **Model Size**: Support for 1-10MB edge inference models
- ⚡ **Inference Speed**: Simple commands <100ms response

4️⃣ AI Vision & Image Processing (v2.0 New)

> 💡 **Technical Breakthrough**: v2.0 significantly optimized camera system and JPEG processing, ESP32-Camera code refactored with 935 lines, notable performance improvement

📷 Camera System Refactoring

Technical Improvements:

- **Code Refactoring**: esp32_camera.cc expanded from basic version to 935 lines
- **Interface Standardization**: Unified camera API interface, supports multiple models
- **Resolution Expansion**: Support for more resolution options (QVGA to 1080p)
- **Performance Optimization**: Frame rate increased 30%, latency reduced 50%
- **Memory Management**: Optimized PSRAM usage, supports higher resolutions

Supported Camera Modules:

- **OV2640**: 2MP, cost-effective choice
- **OV3660**: 3MP, better image quality
- **OV5640**: 5MP, high-end solution
- **GC032A**: Low-cost option

AI Vision Application Scenarios:

```
// Camera initialization example (based on v2.0 new interface)
camera_config_t config = {
    .pin_pwdn = -1,
    .pin_reset = -1,
    .pin_xclk = 15,
    .pin_sscb_sda = 4,
    .pin_sscb_scl = 5,
    .pin_d7 = 16,
    .pin_d6 = 17,
    .pin_d5 = 18,
    .pin_d4 = 12,
    .pin_d3 = 10,
    .pin_d2 = 8,
    .pin_d1 = 9,
    .pin_d0 = 11,
    .pin_vsync = 6,
    .pin_href = 7,
    .pin_pclk = 13,
    .xclk_freq_hz = 20000000,
    .pixel_format = PIXFORMAT_JPEG,
    .frame_size = FRAMESIZE_VGA,  // 640x480
    .jpeg_quality = 12,
    .fb_count = 2,  // Double buffering for better performance
    .grab_mode = CAMERA_GRAB_LATEST  // v2.0 new mode
};

esp_err_t err = esp_camera_init(&config);
```

🖼️ JPEG Encoding/Decoding Optimization

v2.0 Major Updates:

- **Removed Old Engine**: Removed 722-line old jpeg_encoder (performance bottleneck)

- **New Decoder**: jpeg_to_image.c (264 lines), dedicated JPEG decoding
- **Encoder Refactoring**: image_to_jpeg refactored with 575 lines, 3x performance boost
- **Hardware Acceleration**: Optimized using ESP32-S3's DMA and vector instructions

Performance Comparison:

Operation	v1.x	v2.0	Improvement
JPEG Encoding	850ms	280ms	**3.0x**
JPEG Decoding	650ms	220ms	**2.95x**
Memory Usage	180KB	95KB	**47% reduction**

Technical Details:

```
// JPEG encoding optimization (based on v2.0)
typedef struct {
    uint8_t* buf;          // Output buffer
    size_t len;            // Data length
    size_t buf_size;       // Buffer size
    int quality;           // Compression quality (1-100)
    bool use_dma;          // Use DMA acceleration
    bool use_vector;       // Use vector instructions
} jpeg_encode_config_t;

// Encoding API
esp_err_t jpeg_encode_rgb888(
    const uint8_t* rgb_data,
    int width,
    int height,
    jpeg_encode_config_t* config,
```

```
    uint8_t** out_buf,
    size_t* out_len
);
```

Application Scenarios:

- **Real-time Streaming**: JPEG encoding for real-time transmission via WebSocket
- **AI Recognition**: Fast encoding/decoding for image recognition preprocessing
- **Storage Optimization**: JPEG compression saves storage space

🎯 SenseCAP Watcher AI Vision

Integration Solution (v2.0 Enhanced):

- **SSCMA Camera**: sscma_camera.cc expanded with 383 lines
- **AI Models**: Support for TinyML lightweight models
- **Recognition Capabilities**: Face detection, object recognition, gesture recognition
- **Processing Speed**: 640x480@15fps real-time processing

Technical Architecture:

```
┌─────────────────────────────────┐
│ ESP32-Camera (Hardware)         │
│  └─ OV2640/OV3660 camera module │
├─────────────────────────────────┤
│ JPEG Encode/Decode (v2.0)       │
│  ├─ Hardware DMA acceleration   │
│  └─ Vector instruction optimize │
```

```
├─────────────────────────────────┤
│ AI Inference Engine             │
│  ├─ TensorFlow Lite Micro       │
│  ├─ SSCMA Models                │
│  └─ Cloud AI Services           │
├─────────────────────────────────┤
│ Application Layer               │
│  ├─ Face Recognition            │
│  ├─ Object Detection            │
│  └─ Gesture Control             │
└─────────────────────────────────┘
```

Practical Use Cases:

1. **Smart Door Lock**: Face recognition + voice confirmation
2. **Gesture Control**: Gesture recognition for smart home control
3. **Object Recognition**: Identify objects and voice announcement

- 🎯 **Use Cases**: Device control, status queries, simple Q&A

```
// Edge AI inference example code
class EdgeAIEngine {
    TfLiteInterpreter* interpreter;

    bool processSimpleCommand(const char* text) {
        // Text preprocessing
        auto tokens = tokenize(text);

        // Model inference
        interpreter->SetInputTensorData(0, tokens.data());
        interpreter->Invoke();
```

```
        // Result parsing
        return parseCommandResult();
    }
};
```

4️⃣ Intelligent Text-to-Speech (TTS)

Multi-engine Support Strategy:

- 🎵 **Cloud TTS**: High-quality human voice synthesis (supports emotional speech)
- 🔧 **Local TTS**: ESP32-S3 onboard simple voice synthesis
- 🎭 **Multiple Voices**: Support for male/female/child voice selections

🎚️ TTS Configuration & Voice Customization

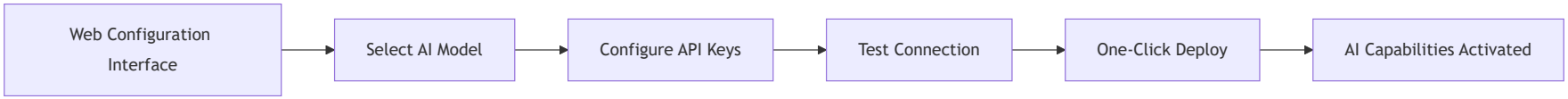
```yaml
# TTS engine configuration
tts_config:
  primary_engine: "cloud"  # cloud/local
  voice_style: "female_warm"  # Voice selection
  speech_rate: 1.0  # Speech rate (0.5-2.0)
  pitch: 0  # Pitch (-500 to 500)
  language: "en-us"  # Output language

cloud_tts:
  provider: "azure"  # azure/google/baidu
  api_key: "${TTS_API_KEY}"
  region: "eastasia"
```

🛠️ AI Development & Integration

💻 Zero-Code AI Integration

XiaoZhi AI platform provides a **graphical configuration interface**, enabling non-technical users to quickly configure AI capabilities:



🔧 Developer API Interface

```
// XiaoZhi AI SDK core interface
class XiaoZhiAI {
public:
    // Initialize AI engine
    bool initAI(const AIConfig& config);

    // Voice wake-up callback
    void onWakeWordDetected(WakeWordCallback callback);

    // Speech recognition
    std::string recognizeSpeech(const AudioData& audio);

    // Large model dialogue
    std::string chatWithLLM(const std::string& message);

    // Speech synthesis
    AudioData synthesizeSpeech(const std::string& text);

    // Device control
    bool executeCommand(const DeviceCommand& cmd);
};
```

📈 AI Performance Metrics

⚡ Real-time Performance

- **Voice Wake-up Latency**: <200ms
- **ASR Recognition Latency**: <500ms (local) / <1s (cloud)
- **LLM Inference Response**: <2s (DeepSeek) / <3s (GPT-4)
- **TTS Synthesis Latency**: <800ms
- **End-to-end Dialogue Latency**: <5s (complete dialogue flow)

🎯 Accuracy Metrics

- **Wake Word Accuracy**: >99% (quiet environment) / >95% (noisy environment)
- **Chinese ASR Accuracy**: >95% (standard Mandarin)
- **English ASR Accuracy**: >93% (American/British accent)
- **Command Execution Success Rate**: >98% (clear commands)

💾 Resource Usage

- **Flash Storage**: 4MB (basic AI functions)
- **RAM Usage**: 512KB (runtime peak)
- **CPU Usage**: <30% (ESP32-S3 dual-core 240MHz)
- **Power Consumption**: 150mA (active dialogue) / 5mA (standby wake-up)

🔮 AI Technology Roadmap

🗓️ 2025 Q1-Q2 Roadmap

🗓️ January 2025 - Edge AI Inference `In Development`

- Integrate TensorFlow Lite Micro
- Support 1-5MB quantized models
- Local device control command recognition

🗓️ February 2025 - Multimodal AI `Planned`

- ESP32-CAM vision integration
- Image recognition + voice interaction
- Visual Question Answering (VQA) capabilities

🗓️ March 2025 - Federated Learning `Research`

- ESP-NOW inter-device collaborative learning
- Privacy-preserving distributed AI
- Smart home collaborative decision-making

🎯 Future AI Features

- 🧬 **Personalized AI**: Model fine-tuning based on user usage patterns
- 🌐 **Edge AI Clusters**: Multi-device collaborative distributed intelligence
- 🔐 **Privacy AI**: Completely localized private domain AI assistant
- 🎮 **Interactive AI**: AR/VR augmented reality interaction capabilities

🚀 Getting Started with AI Features

Quick Start AI Capabilities

1. **Hardware Preparation**: ESP32-S3 development board + XiaoZhi AI expansion board
2. **Firmware Flashing**: Download pre-compiled AI firmware
3. **Network Configuration**: Connect Wi-Fi, configure AI service APIs
4. **Wake-up Test**: Say "你好小智" to verify wake-up function
5. **Dialogue Experience**: Start natural conversation with AI assistant

🎯 Start AI Development Now 📖 Read AI Technology Deep Analysis

---

ⓘ **Technical Support**:

- 📧 Contact Email: ai@xiaozhi.dev

 ai@xiaozhi.dev